Vector Mathematics in Secure RAG

Step-by-Step: From Banking Text to Stored Vector to Retrieved Answer

STEP 0: THE INPUT TEXT

We begin with this raw banking text extracted from a PDF:

> **Original Input Text:**
>
> *"Account Holder: Sai Manoj Yadav, Account Number: 123456789012, IFSC Code: SBIN0001234, Phone: 9876543210, Email: saimanoj@example.com, Transfer Amount: Rs. 45,000, Purpose: Monthly Salary Disbursement"*

This text contains **5 PII entities** that must be protected before vectorization:

#	Entity	Value	Type
1	Account Number	123456789012	Financial
2	IFSC Code	SBIN0001234	Financial
3	Phone Number	9876543210	Personal
4	Email	saimanoj@example.com	Personal
5	Name	Sai Manoj Yadav	Personal

STEP 1: PII DETECTION (Context Lookbehind Algorithm)

The Problem

Both `123456789012` and `9876543210` are digit strings. How does the system mathematically decide which is a phone number and which is an account number?

The Algorithm: 25-Character Sliding Window

For every number found via regex `\b\d{9,18}\b`, the system reads the **25 characters immediately before** the number:

context = text[max(0, match_start - 25) : match_start].lower()

Applied to Our Text

For `123456789012`:

```
25 chars before: "yadav, account number: "
                                ^^^^^^^
Contains "account" → CLASSIFIED AS: ACCOUNT NUMBER ✅
```

For `9876543210`:

```
25 chars before: "bin0001234, phone: "
                              ^^^^^
Contains "phone" → CLASSIFIED AS: PHONE NUMBER ✅
```

For IFSC `SBIN0001234`:

```
Matched by regex: [A-Z]{4}0[A-Z0-9]{6}
Pattern: 4 uppercase letters + 0 + 6 alphanumeric = IFSC format ✅
```

For Email `saimanoj@example.com`:

```
Matched by regex: \S+@\S+
Contains @ symbol = EMAIL format ✅
```

STEP 2: PSEUDONYMIZATION (Token Replacement)

Each detected PII entity is replaced with a UUID-based token:

Original Value	Pseudonym Token	Stored in MongoDB
123456789012	ACCOUNT_f7ce	AES-256 encrypted
SBIN0001234	IFSC_a1b2	AES-256 encrypted
9876543210	PHONE_x9y8	AES-256 encrypted
saimanoj@example.com	EMAIL_m4n5	AES-256 encrypted

Text After Pseudonymization

> **Safe Text (Ready for Vectorization):**
>
> *"Account Holder: Sai Manoj Yadav, Account Number: ACCOUNT_f7ce, IFSC Code: IFSC_a1b2, Phone: PHONE_x9y8, Email: EMAIL_m4n5, Transfer Amount: Rs. 45,000, Purpose: Monthly Salary Disbursement"*

The real PII values no longer exist in the text. Only UUID tokens remain.

STEP 3: TOKENIZATION (Breaking Text into Numbers)

Before the neural network can process text, it must convert English words into numerical **token IDs** using a vocabulary lookup table.

How Tokenization Works

Each word (or sub-word) is mapped to a unique integer from a pre-trained vocabulary of ~30,000 tokens:

```
"Account"     → Token ID: 4070
"Holder"      → Token ID: 17213
":"           → Token ID: 1024
"Sai"         → Token ID: 18392
"Manoj"       → Token ID: 28451
"ACCOUNT"     → Token ID: 4070
"_"           → Token ID: 1035
"f7ce"        → Token ID: 29104
"Transfer"    → Token ID: 5219
"Amount"      → Token ID: 3568
"Salary"      → Token ID: 11247
...
```

Result: Token ID Sequence

```
Input IDs: [101, 4070, 17213, 1024, 18392, 28451, 1010, 4070, 2193, 1024, 4070, 1035, 29104,
1010, ..., 102]

Length: ~45 tokens
101 = [CLS] (start token)
102 = [SEP] (end token)
```

STEP 4: EMBEDDING GENERATION (The Neural Network Mathematics)

The token IDs are fed into the **Gemini Embedding API** (transformer-based neural network). Inside the model, two critical mathematical operations occur:

4.1 Token Embedding Lookup

Each token ID is converted into a **3072-dimensional vector** by looking up a pre-trained embedding matrix **E**:

$$e_i = E[\ token_id_i\ ]$$

Where E is a matrix of size (30000 × 3072)

Each row is a 3072-dimensional vector representing one word

Example

```
"Account" (ID: 4070) → E[4070] = [0.0234, -0.0891, 0.0412, ..., 0.0156]  (3072 values)
"Salary"  (ID: 11247) → E[11247] = [-0.0312, 0.0567, 0.0823, ..., -0.0234] (3072 values)
```

4.2 Self-Attention Formula

The transformer computes **how much attention** each word should pay to every other word using the **Scaled Dot-Product Self-Attention** equation:

$$\text{Attention}(Q, K, V) = \text{softmax}(\ Q \times K^T / \sqrt{d_k}\ ) \times V$$

What Each Variable Means

Symbol	Name	Meaning	Size
Q	Query	"What am I looking for?" for each word	$(n \times d_k)$
K	Key	"What properties do I have?" for each word	$(n \times d_k)$
V	Value	"What information do I carry?"	$(n \times d_v)$
$\mathbf{d_k}$	Key Dimension	Scaling factor (prevents exploding gradients)	256
n	Sequence Length	Number of tokens in input	~45

How Q, K, V Are Calculated

Each is derived from the token embeddings by multiplying with learned weight matrices:

$$Q = X \times W_Q \quad | \quad K = X \times W_K \quad | \quad V = X \times W_V$$

Where X is the matrix of all token embeddings stacked together.

Step-by-Step Calculation

Step A: Compute attention scores

```
scores = Q × K^T
       = (45 × 256) × (256 × 45)
       = (45 × 45) matrix of raw attention scores
```

Step B: Scale by $\sqrt{d_k}$

```
scaled_scores = scores / √256 = scores / 16

Why? Without scaling, the dot products grow very large for high dimensions,
causing softmax to produce near-zero gradients (vanishing gradient problem).
```

Step C: Apply softmax (convert to probabilities)

```
attention_weights = softmax(scaled_scores)

Each row now sums to 1.0 — representing how much attention
each word gives to every other word.
```

Step D: Multiply by Values

```
output = attention_weights × V
       = (45 × 45) × (45 × 256)
       = (45 × 256) — contextualized embeddings
```

4.3 Multi-Head Attention

The model runs **12 attention heads in parallel**, each learning different relationships:

Head	What It Might Learn
Head 1	"Account" relates to "Number" (syntactic)
Head 2	"Salary" relates to "Transfer" (semantic)
Head 3	"Rs. 45,000" relates to "Amount" (numeric)
Head 4	"ACCOUNT_f7ce" is an entity token (pattern)

All 12 heads are concatenated and projected:

$$\text{MultiHead}(Q,K,V) = \text{Concat}(\text{head}_1, \text{head}_2, \ldots, \text{head}_{12}) \times W_O$$

4.4 Final Output: The Document Vector

After all transformer layers process the tokens, the model produces a **single 3072-dimensional vector** representing the entire document's meaning:

> **Document Vector (before noise):**
>
> `[-0.008764, 0.011886, 0.022306, -0.082017, 0.018183, -0.000602, 0.008421, -0.007880, 0.015234, -0.031456, 0.044891, 0.002347, ..., -0.019234]`
>
> **Total: 3072 floating-point numbers**
> Each number represents one dimension of the document's semantic meaning.

STEP 5: DIFFERENTIAL PRIVACY (Laplacian Noise Injection)

Why Add Noise?

Without noise, an attacker could run the vector **backward** through the model to reconstruct the original English text. This is called an **Embedding Inversion Attack**. Noise mathematically destroys the reverse path.

The Laplace Distribution

The noise is sampled from the **Laplace (double exponential) distribution**:

$$f(x \mid \mu, b) = (1 / 2b) \times \exp(-|x - \mu| / b)$$

Parameter	Value	Meaning
μ (mu)	0	Center of noise — we add AND subtract equally
b (scale)	Δf / ε	Controls noise intensity
Δf (sensitivity)	~0.01	Max change per data point
ε (epsilon)	~1.0	Privacy budget — lower = more secure

The ε-Differential Privacy Mechanism

$$M(x) = f(x) + Lap(\Delta f / \varepsilon)$$

For each of the 3072 coordinates:

$$v'_i = v_i + noise_i$$

where $noise_i \sim Laplace(0, \Delta f/\varepsilon)$

Applied to Our Vector

```
Dimension 1:   -0.008764  +  0.001153  =  -0.007611
Dimension 2:    0.011886  + -0.002114  =   0.009772
Dimension 3:    0.022306  +  0.000294  =   0.022600
Dimension 4:   -0.082017  +  0.003561  =  -0.078456
Dimension 5:    0.018183  + -0.001103  =   0.017080
...
Dimension 3072: -0.019234  +  0.000847  =  -0.018387
```

> **Key Insight:** The noise shifts each coordinate by approximately ±0.003. This is small enough that the vector still "points in the same direction" (preserving search accuracy), but large enough that precise floating-point values cannot be reversed into English words.

Before vs After Noise

Original: [-0.008764, 0.011886, 0.022306, -0.082017, 0.018183, ...] **DP-Noised:** [-0.007611, 0.009772, 0.022600, -0.078456, 0.017080, ...] **Shift:** [+0.001153, -0.002114, +0.000294, +0.003561, -0.001103, ...]

STEP 6: STORAGE IN PINECONE

The noised vector is stored in Pinecone as a geometric coordinate point:

Pinecone Record:

```
{
  "id": "9646249b-e311-4def-819f-0a2023c7e4ad",
  "values": [-0.007611, 0.009772, 0.022600, ..., -0.018387],   ← 3072
floats
  "metadata": {
    "text": "Account Holder: Sai Manoj Yadav, Account Number: ACCOUNT_f7ce,
             IFSC Code: IFSC_a1b2, Phone: PHONE_x9y8, Email: EMAIL_m4n5,
             Transfer Amount: Rs. 45000, Purpose: Monthly Salary
Disbursement",
    "owner_id": "sai"
  }
}
```

The text stored in metadata is **pseudonymized** (no real PII). The vector values are **DP-noised** (no inversion possible). The `owner_id` enforces **multi-tenant isolation**.

STEP 7: RETRIEVAL (How a Question Finds the Answer)

The Question

User "sai" asks: **"What is the transfer amount?"**

7.1 Question → Vector

The question text goes through the **same** Gemini Embedding API (but with `task_type="RETRIEVAL_QUERY"`):

```
Question Vector (Q): [0.012450, -0.034521, 0.045123, -0.056789, 0.023456, ..., 0.008912] (3072 dimensions)
```

7.2 Cosine Similarity Calculation

Pinecone compares the question vector **Q** against every stored document vector **D** using **Cosine Similarity**:

$$\cos(\theta) = (Q \cdot D) / (||Q|| \times ||D||)$$

Breaking Down the Formula

Numerator: Dot Product (Q · D)

```
Q · D = Q₁×D₁ + Q₂×D₂ + Q₃×D₃ + ... + Q₃₀₇₂×D₃₀₇₂

      = (0.012450 × -0.007611)
      + (-0.034521 × 0.009772)
      + (0.045123 × 0.022600)
      + ...
      + (0.008912 × -0.018387)

      = 0.8234  (example result)
```

Denominator: Product of Magnitudes

```
||Q|| = √( Q₁² + Q₂² + Q₃² + ... + Q₃₀₇₂² )
      = √( 0.012450² + 0.034521² + 0.045123² + ... )
      = 0.9156  (example)

||D|| = √( D₁² + D₂² + D₃² + ... + D₃₀₇₂² )
      = √( 0.007611² + 0.009772² + 0.022600² + ... )
      = 0.9023  (example)
```

Final Cosine Similarity

```
cos(θ) = 0.8234 / (0.9156 × 0.9023)
       = 0.8234 / 0.8262
       = 0.9966
```

7.3 Interpreting the Score

Cosine Score	Meaning	Action
1.00	Identical meaning	Perfect match
0.9966	**Very high similarity**	✅ **Return this document**
0.70	Moderately related	Maybe relevant
0.50	Weak connection	Below threshold (0.5), reject
0.00	Completely unrelated	Ignore

> **Result:** Cosine similarity = 0.9966 ≥ threshold (0.5)
> → Our banking document is returned as the **top match** for "What is the transfer amount?"

7.4 Owner-ID Filtering

Before the cosine calculation even runs, Pinecone applies a **metadata filter**:

filter = { "owner_id": { "$in": ["admin", "sai"] } }

- Documents owned by "sai" → ✅ Included in search
- Documents owned by "admin" → ✅ Included (shared knowledge base)
- Documents owned by "ravi" → ❌ **Mathematically excluded** — cosine is never calculated

---

STEP 8: DE-PSEUDONYMIZATION (Owner-Only Decryption)

The LLM Response (Before De-pseudonymization)

```
"The transfer amount is Rs. 45,000 for account ACCOUNT_f7ce."
```

AES-256 Decryption (Owner Verification)

The backend checks: **Is the requester ("sai") the same as the owner_id in the mapping?**

Ciphertext = AES_DECRYPT(encrypted_blob, KEY, IV)

KEY = SHA-256(SECRET_KEY) → 32 bytes (256 bits)
IV = first 16 bytes of stored blob
Mode = CBC (Cipher Block Chaining) with PKCS7 padding

```
MongoDB lookup: { token: "ACCOUNT_f7ce", owner_id: "sai" }
Found → original (encrypted): "xK9$#mZ!qR2...base64..."
AES decrypt → "123456789012"
Replace in response: ACCOUNT_f7ce → 123456789012
```

Final Response to User "sai"

> **"The transfer amount is Rs. 45,000 for account 123456789012."**

If User "ravi" Somehow Got the Same Response

> **"The transfer amount is Rs. 45,000 for account ACCOUNT_f7ce."**
>
> MongoDB lookup: { token: "ACCOUNT_f7ce", owner_id: "ravi" } → **NOT FOUND**
> Token stays as-is. Ravi never sees the real account number.

MATHEMATICAL SUMMARY

Step	Formula / Algorithm	Purpose
PII Detection	`context = text[start-25:start]`	25-char lookbehind classification
Pseudonymization	`text.replace(value, UUID_token)`	Remove real PII from text
AES-256 Encryption	`AES_CBC(plaintext, key, iv)`	Encrypt mapping in MongoDB
Tokenization	`token_id = vocab[word]`	Convert words to integers
Embedding Lookup	`e = E[token_id]`	Convert integers to 3072-dim vectors
Self-Attention	$\mathrm{softmax}(QK^T / \sqrt{d_k}) \times V$	Contextualize word relationships
Multi-Head	$\mathrm{Concat}(\mathrm{head}_1 \ldots \mathrm{head}_{12}) \times W_O$	12 parallel attention perspectives
DP Noise	$v' = v + \mathrm{Lap}(\Delta f/\varepsilon)$	Prevent embedding inversion attacks
Storage	`Pinecone.upsert(vector, metadata)`	Store with owner_id tag
Retrieval	$\cos(\theta) = (Q \cdot D) / (\lVert Q \rVert \times \lVert D \rVert)$	Find semantically similar documents
De-pseudonymize	`AES_DECRYPT(blob, key, iv)`	Owner-only PII recovery

📐 **11 Mathematical Operations × 3072 Dimensions × AES-256 Encryption**

Complete End-to-End Vector Pipeline — From English Text to Secure Retrieval